TECHNICAL DOCUMENTATION · INTERNAL

School Management System

Project Flow Documentation

DOCUMENT TYPE
End-to-end process, architecture & module flow

VERSION
1.0

EFFECTIVE DATE
08 October 2026

STACK
Laravel 12 API · React 19 SPA · MySQL

AUDIENCE
Project stakeholders, developers, QA, operations

CLASSIFICATION
Internal — project delivery use

Document control

Item	Detail
Title	School Management System — Project Flow Documentation
Purpose	Describe how the product is structured, how users move through it, and how data and requests flow from UI to database.
Scope	Live application as implemented: staff console, parent portal, REST API v1, roles, modules, deployment.
Related files	`README.md`, `DEPLOYMENT.md`, `backend/routes/api.php`, `backend/config/permissions.php`
Local URLs (dev)	Frontend `http://localhost:5173` · API `http://localhost:8000/api/v1`
Database	`school_management` (MySQL / XAMPP)

Contents

1 Executive summary

2 Project purpose, objectives and scope

3 Stakeholders and user roles

4 Technology stack

5 System architecture

6 MVC design (backend and frontend)

7 Project delivery lifecycle

8 Environment and setup flow

9 Authentication and session flow

10 Authorization and permission matrix

11 Application navigation and routing

12 Module process flows

13 Parent portal flow

14 API request lifecycle

15 Data architecture

16 Cross-cutting services (notify, pay, audit, files)

17 Security, testing and operations

18 Production deployment flow

A Appendix — demo accounts, directory map, glossary

1. Executive summary

The School Management System is a web application that runs the daily operations of a school: student records, teaching, attendance, examinations, fees, communication, library, transport and HR. Staff work in a permission-based console. Parents use a separate portal for children, fees, notices, messages and calendar.

The product is split into two applications that talk over a versioned REST API. The backend is Laravel (PHP 8.2+) with Laravel Sanctum tokens. The frontend is a React 19 single-page app structured in an MVC-style layout (models, controllers, views). MySQL stores operational data.

4	18+	API v1	MVC
USER ROLES	FUNCTIONAL MODULES	BEARER TOKEN REST	BOTH TIERS

> **How to read this document.** Sections 5–6 describe structure. Sections 9–14 describe runtime flow (login → permission check → module work → parent portal). Sections 15–18 describe data, security and go-live. Process boxes show the order of work, not optional marketing features.

2. Project purpose, objectives and scope

2.1 Purpose

Replace fragmented paper and spreadsheet processes with one system of record for academics, finance and parent communication, with clear role boundaries so teachers cannot post office payments and parents cannot edit school master data.

2.2 Objectives

- Maintain a single student, class, subject and teacher directory.
- Capture attendance and exam grades against class/subject context.
- Assign and collect fees (counter payment and online intent/confirm).
- Publish notices, messages and calendar events to the right audience.
- Support library loans, transport assignment, HR leave/payroll, and academic artefacts (assignments, quizzes, syllabus, promotions).
- Record who changed what (audit log) and who was notified (notification log).

2.3 In scope

Staff SPA routes under `/dashboard` … `/audit`; parent SPA under `/parent`; REST endpoints under `/api/v1`; MySQL schema via Laravel migrations; queued email alerts; PDF reports; CSV exports; file uploads on the public disk; demo seed data.

2.4 Out of scope (this release)

Native mobile apps, full live payment-gateway webhooks beyond demo/simulated checkout, multi-tenant multi-school hosting, and student self-service login (students are records; parents and staff are users).

3. Stakeholders and user roles

Role	Who	Primary workspace	Typical outcomes
admin	School administrator	Full staff console	All abilities (*). Configure master data, users, finance, HR, audit.
teacher	Faculty (linked `teachers.user_id`)	Staff console (filtered nav)	Attendance, exams/grades, academics, library, messaging, reports.
staff	Front office / accounts	Staff console (office modules)	Parents, fees, notices, transport, HR, promotions, expenses, audit. No grade edits.
parent	Guardian with portal user	Parent portal only	Children overview, fee pay, notices, messages, calendar. No staff APIs.

Permissions live in `backend/config/permissions.php`. Frontend navigation hides links the user cannot use (`STAFF_NAV_LINKS` + permission keys).

4. Technology stack

Layer	Technology	Responsibility
Presentation	React 19, Vite 6, React Router 7	SPA screens, client routing, token session in `localStorage`
Client structure	`src/views`, `src/controllers`, `src/models`	UI / state hooks / HTTP API wrappers
API	Laravel 12, PHP 8.2+	Validation, authorization, business rules, JSON resources
Auth	Laravel Sanctum	Personal access tokens, `Authorization: Bearer`
Data	MySQL (utf8mb4), Eloquent	Transactional school data
Jobs / mail	Laravel queue + `SchoolAlertMail`	Fee reminders, absence alerts, announcement notices
PDF	barryvdh/laravel-dompdf	Report card, attendance sheet, fee receipt
Files	Laravel `public` disk	Student photos, documents, notice attachments
Quality	PHPUnit feature tests	`ApiSchoolManagementTest` against in-memory SQLite

5. System architecture

The runtime is a classic SPA + API split. The browser never talks to MySQL. Every mutation goes through Laravel controllers, Form Requests where used, Eloquent models, and JSON resources.

LAYER 1 — USERS
Admin · Teacher · Office staff · Parent (browser)

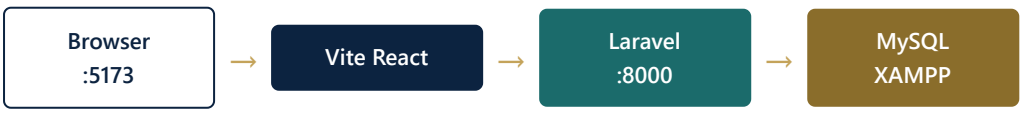
LAYER 2 — REACT SPA (VITE)
Login / Register · Staff AppLayout · ParentLayout · Views + Controllers + Models · `ApiClient.js`

LAYER 3 — HTTP
REST `/api/v1/*` · JSON · Bearer token · CORS via `FRONTEND_URL`

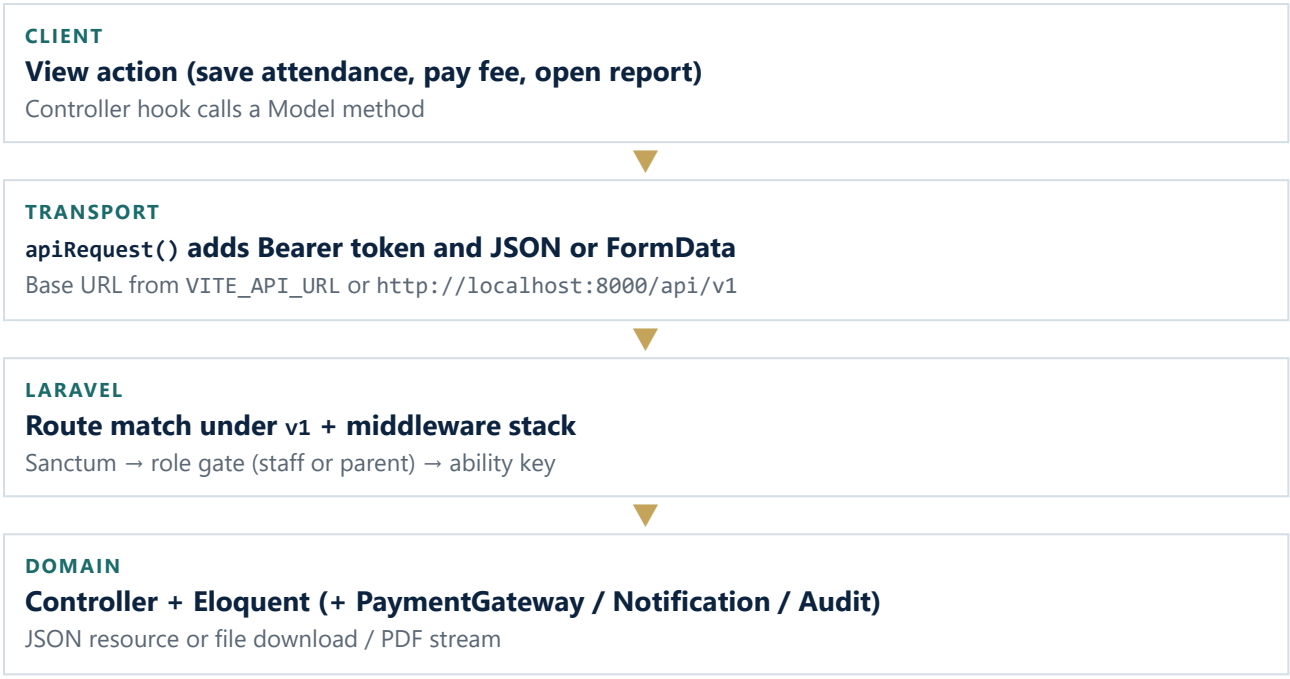
LAYER 4 — LARAVEL API
Routes → Middleware (auth, staff/parent, permission) → Controllers → Services / Models → Resources

LAYER 5 — PERSISTENCE & SIDE EFFECTS
MySQL · Queue / Mail · Storage (`storage/app/public`) · Audit & notification logs · Backups

5.1 Logical deployment (development)



5.2 High-level request path



6. MVC design (backend and frontend)

Both tiers follow a Model–View–Controller split so screens stay thin, HTTP stays in one place, and Laravel remains the authority for rules.

Backend (Laravel)

- **Model** — `app/Models` (Student, Exam, StudentFee, …)
- **Controller** — `app/Http/Controllers/Api`
- **View / output** — API Resources + Blade mail + DomPDF
- **Requests** — validation at the HTTP edge
- **Services** — PaymentGateway, SchoolNotification
- **Support** — Permission, AuditLogger

Frontend (React MVC-style)

- **View** — `src/views/**` (screens, layout, forms)
- **Controller** — `src/controllers` (Auth, hooks)
- **Model** — `src/models` (API wrappers per domain)
- **Config** — `navigation.js`, env API URL
- **Guards** — ProtectedRoute, AdminRoute, ParentRoute

6.1 Frontend collaboration pattern



Example: `AttendanceView` uses `useAttendanceController` which calls `AttendanceModel`. Generic CRUD screens use `useResourceController`.

7. Project delivery lifecycle

Work on this codebase follows a linear delivery path with repeatable local setup and a production cut-over described in `DEPLOYMENT.md`.

PHASE 1

Discover & scope

Roles (admin/teacher/staff/parent), modules, and permission keys agreed as the product contract.

PHASE 2

Data design

Migrations under `backend/database/migrations` — core directory first, then academics, fees, comms, library, transport, HR, production-readiness (audit, documents).

PHASE 3

API implementation

Controllers + middleware aliases in `bootstrap/app.php` (`admin`, `staff`, `parent`, `permission`).

PHASE 4

SPA implementation

Routes in `App.jsx`, nav in `navigation.js`, domain models for each module.

PHASE 5

Seed, test, harden

`SchoolManagementSeeder`, PHPUnit API tests, audit log, backups, HTTPS/CORS production flags.

PHASE 6

Release & operate

Build SPA, cache config/routes, queue worker, `storage:link`, `php artisan school:backup`.

8. Environment and setup flow

8.1 First-time local bring-up

DATABASE

Create `school_management` (utf8mb4)

BACKEND

`composer install` → **configure** `backend/.env` → `php artisan migrate --seed` → `php artisan serve`

FRONTEND

`npm install` → **optional** `frontend/.env` **from example** → `npm run dev`

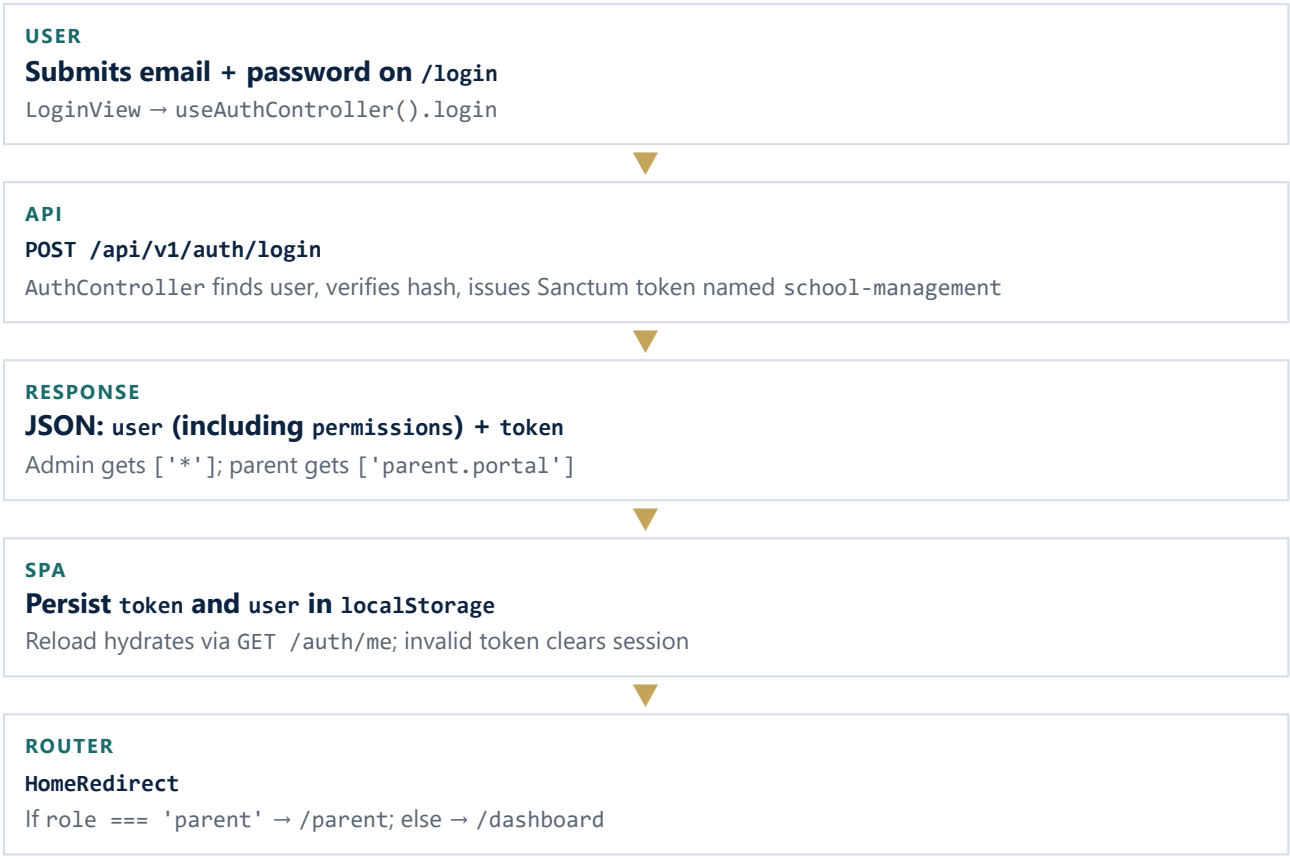
VERIFY

Open SPA, sign in with a seeded role, confirm dashboard or parent home loads.

Prerequisites: PHP 8.2+, Composer, MySQL (XAMPP), Node.js 20+.

9. Authentication and session flow

9.1 Login (happy path)



9.2 Register

`POST /auth/register` creates a user with role **admin** (bootstrap account). Password min 8 characters, must be confirmed. A token is returned the same way as login.

9.3 Logout

`POST /auth/logout` (authenticated) deletes the current access token. The SPA always clears local storage even if the API call fails.

9.4 Failure path

Bad credentials raise Laravel validation on `email` ("The provided credentials are incorrect."). The SPA surfaces the message; no token is stored.

10. Authorization and permission matrix

After Sanctum authenticates the user, two middleware layers apply:

1. **Role lane** — `staff` (admin, teacher, staff) versus `parent` (parent portal prefix only).
2. **Ability** — `permission:ability.key` checked by `EnsurePermission` using `Permission::allows()`.

Parents always fail staff ability checks (staff APIs return 403). Admin abilities are wildcard `*`.

Capability	Admin	Teacher	Staff	Parent
Dashboard / directory read	Yes	Yes	Yes (limited)	Portal only
Manage students / classes / teachers	Yes	View (not full manage)	View students/classes	No
Attendance mark	Yes	Yes	No	No
Exams & grades	Yes	Yes	No	No
Fees, payments, fee types	Yes	No	Yes	Pay own invoices
Announcements manage	Yes	Feed only	Yes	Read targeted
Academics (assign/quiz/syllabus)	Yes	View + manage	View	No
Bulk promotion	Yes	No	Yes	No
Library	Yes	View + manage	Manage	No
Transport / HR / expenses / audit	Yes	No	Yes	No
Messages	Yes	View + send	View + send + moderate	Inbox / reply
Notify (fee / absence / notice)	Yes	No	Yes	Receive

> Denied API calls return HTTP 403 with `You do not have permission for this action.` The SPA should not offer those actions in the nav, but the API remains the source of truth.

11. Application navigation and routing

11.1 Route tree

Path	Guard	Layout	Audience
`/login`, `/register`	Public	—	All
`/`	Authenticated	—	Redirect by role
Staff modules (`/dashboard` … `/audit`)	Protected + AdminRoute	AppLayout	admin, teacher, staff
`/parent` and children	Protected + ParentRoute	ParentLayout	parent
Unknown path	—	—	Same as home redirect

11.2 Staff modules (navigation order)

Dashboard · Students · Teachers · Classes · Subjects · Attendance · Exams & grades · Academics · Timetable · Parents · Fees · Finance · Audit log · Notices · Communication · Reports · Library · Transport · Staff / HR

11.3 Parent modules

Home (overview) · Fees · Announcements · Messages · Calendar

11.4 Cross-lane protection

A parent hitting a staff URL is sent to `/parent`. A staff user hitting parent URLs is sent to `/dashboard`. This mirrors the API split between `middleware('staff')` and `middleware('parent')`.

12. Module process flows

Each subsection is the operational sequence a school would follow. Actors and APIs are those implemented in this repository.

12.1 Master data — classes, teachers, students, subjects



Students carry a unique admission number and optional class. Subjects belong to a class and optionally a teacher. Photos/documents: `POST students/{id}/photo` and `/documents` (ability `files.upload`).

12.2 Parent–student linking

OFFICE / ADMIN

Create parent guardian (optional portal user)

`POST /parents` — ability `parents.manage`

▼

LINK

Attach students with relation on pivot `parent_student`

▼

PARENT LOGIN

Portal scoped to linked children only

Overview, fees, notices, messages, calendar, online pay

12.3 Daily attendance

TEACHER / ADMIN

Open attendance sheet for class + date

`GET /attendance/sheet`

▼

MARK

Present / absent (and related statuses) in bulk

`POST /attendance/bulk` — `attendance.manage`

▼

REVIEW

Summary and history

`GET /attendance/summary`, list endpoint; dashboard “marked today”

▼

OFFICE (OPTIONAL)

Send absence alerts to parents

`POST /communication/notifications/absence-alerts`

12.4 Timetable



Teachers may view the grid; managing slots requires `timetable.manage` (admin).

12.5 Examinations and grades

TEACHER / ADMIN

Create exam (code, class/subject, date, max marks)

`POST /exams`

▼

MARKING

Load sheet → enter marks in bulk

`GET /exams/{id}/sheet` then `POST /exams/{id}/grades/bulk`

▼

INSIGHT

Exam summary + dashboard pass-rate chart (≥ 50% of max marks)

PDF report card via Reports module

12.6 Academics — assignments, quizzes, syllabus, lesson plans, promotion

Work item	Actor	Flow
Assignment	Teacher	Create → students submit → teacher patches submission (grade/feedback)
Online quiz	Teacher / student-attempt APIs	Create quiz + questions → start attempt → submit answers → review attempts
Syllabus / lesson plan	Teacher	CRUD under `/academics/syllabi` and `/lesson-plans`
Promotion	Staff / admin	`POST /academics/promotions/bulk` moves a class cohort; history via GET

12.7 Fees (office collection)

ACCOUNTS

Define fee types

`/fee-types` — `fee_types.manage`

▼

ASSIGN

Invoice a student or bulk-assign a class

`POST /student-fees` or `/student-fees/bulk-class`

▼

ADJUST

Optional discount

`PATCH /student-fees/{id}/discount`

▼

COLLECT

Record counter payment

`POST /student-fees/{id}/payments` — status pending → partial → paid

▼

FOLLOW-UP

Fee reminders + PDF receipt + finance expense tracking

12.8 Online payment (parent)

Open invoice → payment-intent → Gateway order → confirm → FeePayment + paid

`PaymentGatewayService` supports `razorpay`, `stripe`, or `simulated`. Confirm runs in a DB transaction: order → `FeePayment` (method online) → refresh fee totals. Nothing is charged if balance due is zero.

12.9 Notices and communication

Announcements

CRUD + optional attachment. Feed for teachers. Parents see published items targeted to their children's classes. Staff can fan-out email/SMS-style notices via notification APIs.

Threads & calendar

Staff create/reply/moderate message threads. Parents list threads, open one, reply. Calendar events are created by staff; parents read the school calendar.

12.10 Library

Catalogue book → Issue loan → Return loan

12.11 Transport

Route → Bus → Assign student → Remove assignment

12.12 HR

Staff directory CRUD → leave request create/update (approve/reject style patch) → payroll entries. Ability split: `hr.view` vs `hr.manage`.

12.13 Reports, exports, dashboard, finance, audit

Surface	Flow
Dashboard	`GET /dashboard` — counts + 7-day attendance trend, 6-month fee collection, recent exam pass rates
PDF reports	Report card, attendance sheet, fee receipt (DomPDF)
CSV exports	Students, attendance, fees (`exports.download`)
Finance	Expense CRUD + summary; list online payment orders
Audit	`GET /audit-logs` — actor, action, model, old/new values, IP

13. Parent portal flow

The parent never uses staff controllers. All reads are filtered by the authenticated user's `parentProfile` and linked students. Missing profile returns 404.

LOGIN
Role parent → `/parent`

▼

OVERVIEW
`GET /parent/overview`
Children + fee due summary (pending/partial balances)

▼

FEES
`GET /parent/fees` → optional payment-intent → confirm

▼

SCHOOL LIFE
Announcements, messages (get/reply), calendar, notification history

Parent APIs: `/parent/overview`, `/fees`, `/announcements`, `/messages`, `/calendar`, `/notifications`, payment-intent and confirm.

14. API request lifecycle

14.1 Prefix

All application APIs are grouped under `/api/v1` (Laravel `routes/api.php` plus the framework `/api` prefix).

14.2 Middleware pipeline (staff example)



14.3 Typical controller contract

1. Validate input (Form Request or inline rules).
2. Authorize already enforced by middleware.
3. Mutate Eloquent models; call a service for payments/notifications.
4. Optionally write `AuditLogger`.
5. Return Resource JSON, paginated collection, PDF, or CSV.

14.4 Client error handling

`ApiClient` parses JSON errors into `ApiError` (message, HTTP status, validation bag). File downloads use a separate blob helper with the same Bearer header.

15. Data architecture

15.1 Core academic graph

User (role) → Teacher (optional user_id) · SchoolClass (class_teacher_id) → Student (school_class_id) → Subject (class + teacher) → Attendance, Exam/ExamGrade, TimetableSlot.

15.2 Family and money

ParentGuardian (optional user_id) ↔ Student (pivot relation) → StudentFee (fee type, amounts, status) → FeePayment · OnlinePaymentOrder.

15.3 Supporting domains

Domain	Principal tables / models
Communication	Announcement, MessageThread, Message, CalendarEvent, NotificationLog
Academics extra	Assignment, AssignmentSubmission, OnlineQuiz (+ questions/attempts/answers), Syllabus, LessonPlan, StudentPromotion
Library	Book, BookLoan
Transport	TransportRoute, Bus, StudentTransportAssignment
HR	StaffMember, LeaveRequest, PayrollEntry
Ops	Expense, AuditLog, StudentDocument, jobs/cache (Laravel), personal_access_tokens

15.4 Identity rules

- Students are not login users; parents and staff are `users`.
- Teachers can be linked to a user for the teacher role.
- Admission numbers and many business codes are unique.

16. Cross-cutting services

16.1 Notifications

`SchoolNotificationService` writes `NotificationLog`, then emails via `SchoolAlertMail` (queued when the queue is not `sync`). SMS is logged as simulated send. Batch helpers: fee reminders (open balances), absence alerts, announcement notices.

16.2 Audit

`AuditLogger::log` stores actor, action, auditable type/id, old/new values, and client IP. Intended for sensitive office mutations (fees, promotions, HR, etc.).

16.3 Files

Public disk + `php artisan storage:link`. Student photo URL is appended on the Student model. Announcements may carry an attachment upload.

16.4 Backups

`php artisan school:backup` writes a manifest under `storage/backups`; `--storage` also zips public uploads. Host-level `mysqldump` remains recommended in production.

17. Security, testing and operations

17.1 Security controls

- Passwords hashed with Laravel's hasher; tokens are Sanctum personal access tokens.
- Role lanes prevent parent tokens from calling staff routes and vice versa.
- Fine-grained abilities stop teachers from recording fees and staff from posting grades.
- Production: `APP_DEBUG=false`, `FORCE_HTTPS`, CORS allow-list, Sanctum stateful domains.
- Uploads constrained to authenticated file abilities; serve via storage link, not raw disk paths.

17.2 Testing

From `backend/`: `php artisan test --filter=ApiSchoolManagementTest`. Tests use in-memory SQLite and seed demo data — suitable for regression of auth, CRUD and core school flows.

17.3 Runtime operations

Concern	Practice
Queue worker	`php artisan queue:work --tries=3` (Supervisor/systemd in production)
Logs	Laravel log + notification/audit tables
Health	Laravel `/up`

18. Production deployment flow

CONFIGURE
Copy env examples; set production URL, CORS, mail, queue, frontend API URL

▼

BACKEND
`composer install --no-dev` → key → migrate → optional seed → `storage:link` → config/route cache

▼

FRONTEND
`npm ci` && `npm run build` → serve `frontend/dist` with SPA fallback to `index.html`

▼

EDGE
Terminate TLS; set `FORCE_HTTPS=true`; trusted proxies if needed

▼

RUN
Queue worker + scheduled DB dumps + `school:backup`

Appendix A — Demo accounts

Seeded by `SchoolManagementSeeder`. Password for all: `password`.

Role	Email	Lands on
Admin	`admin@school.local`	Staff dashboard
Teacher	`teacher@school.local`	Staff dashboard (academic nav)
Staff	`staff@school.local`	Staff dashboard (office nav)
Parent	`parent@school.local`	Parent portal

Illustrative seed story: Grade 10-A, teacher Jane Smith, student John Doe (STU-1001), parent Robert Doe, Mathematics MATH-10A, plus sample attendance, exam, fees, notices and related records.

Appendix B — Repository map

Path	Role in the flow
`backend/routes/api.php`	Canonical API surface and middleware
`backend/config/permissions.php`	Role → ability contract
`backend/app/Http/Controllers/Api`	Use-case entry points
`backend/app/Services`	Payments and notifications
`backend/database/migrations`	Schema evolution
`frontend/src/App.jsx`	SPA route tree and guards
`frontend/src/config/navigation.js`	Staff menu vs permissions
`frontend/src/models`	HTTP adapters per module
`DEPLOYMENT.md`	Production checklist

Appendix C — Glossary

Term	Meaning in this project
Ability / permission key	String such as `fees.manage` checked by middleware
Staff lane	Users with role admin, teacher, or staff using the office/academic SPA
Parent lane	Users with role parent using `/parent/*` APIs only
Sanctum token	Bearer credential stored in the browser after login
Bulk sheet	Class-dated grid for attendance or exam marks saved in one request
Open fee	StudentFee status pending or partial (balance remaining)
Simulated gateway	Demo online pay path that still writes a real FeePayment on confirm

End of document. This PDF is the project flow reference for the School Management System as implemented on 08 October 2026. Behavioural source of truth remains the running code under `backend/` and `frontend/`.